

아키텍처 — 두 개의 데이터 흐름

이 문서는 시스템을 읽기 경로(사용자가 텔레그램으로 라벨 사진을 보내 조회)와 쓰기 경로(관리자가 웹에서 엑셀을 올려 DB를 갱신) 두 흐름으로 나누어 정리합니다. 각 단계에서 데이터가 어떤 형태로 변환되는지와 어떤 기술이 그 변환을 담당하는지에 초점을 맞춥니다.

- 관련 문서
- `SETUP.md` — 실제 배포/설정 절차
 - `PORTFOLIO_GUIDE.md` — 기능별 상세 설명과 구현 배경
 - `../database/` — SQL 스키마 및 마이그레이션

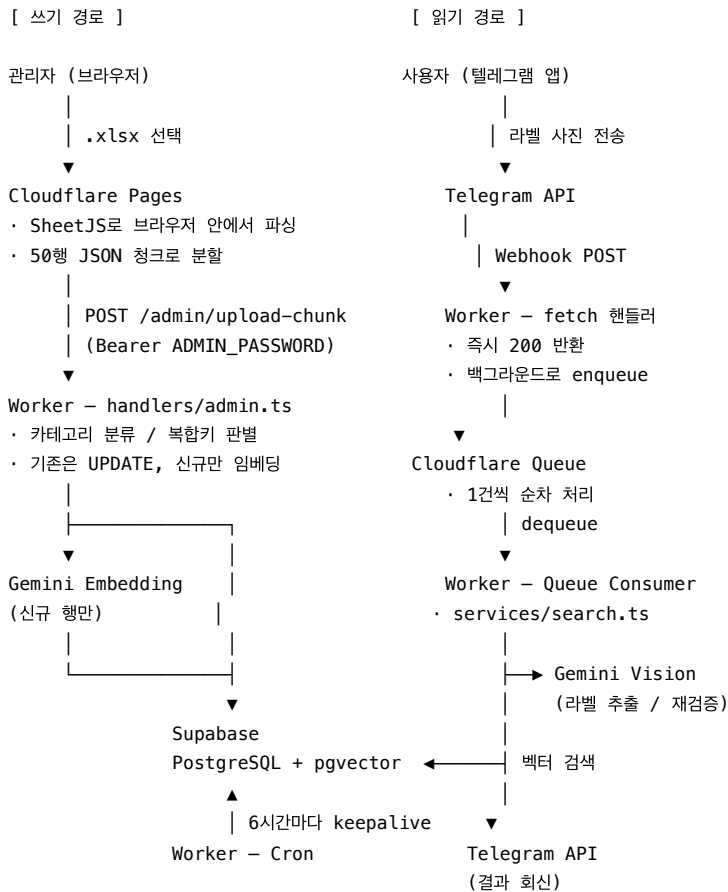
1. 구성 요소

시스템은 아래 6개 구성 요소로 이루어져 있습니다. 이 가운데 독립적으로 배포되는 것은 관리자 웹과 백엔드 둘뿐이고, 나머지는 관리형 서비스이거나 백엔드에 딸린 바인딩입니다. 구성 요소끼리는 공유 런타임이나 공유 파일시스템 없이 HTTP로만 주고받습니다.

구성 요소	실체	배포 단위	소스
관리자 웹	정적 HTML + Vanilla JS	Cloudflare Pages (<code>*.pages.dev</code>)	<code>admin/</code>
백엔드	Hono on Cloudflare Workers	<code>*.workers.dev</code>	<code>backend/src/</code>
비동기 큐	Cloudflare Queues	Worker 바인딩	<code>wrangler.toml</code>
데이터베이스	Supabase PostgreSQL + pgvector	관리형	<code>database/*.sql</code>
AI	Google Gemini (Vision + Embedding)	외부 API	<code>services/gemini.ts</code>
사용자 접점	Telegram Bot API	외부 API	<code>services/telegram.ts</code>

중요: 관리자 웹과 Worker는 서로 다른 배포입니다. Pages 사이트가 Worker를 호스팅하는 게 아니라, 브라우저에서 Worker의 `/admin/*` 엔드포인트를 `fetch` 로 호출합니다. 그래서 Worker에 `*.pages.dev` 만 허용하는 CORS 설정이 존재합니다 (`backend/src/index.ts:12`).

전체 구성도



두 경로가 만나는 지점은 **Supabase의 name_embedding 벡터 컬럼 하나**입니다.
쓰기 경로가 텍스트를 768차원 벡터로 바꿔 넣고, 읽기 경로가 이미지를 같은 768차원 벡터 공간으로 투영해 검색합니다. **양쪽 모두 같은 임베딩 모델을 써야만** 검색이 성립합니다 (gemini-embedding-001 , outputDimensionality: 768).
다만 벡터로 만드는 **텍스트의 재료는 양쪽이 다릅니다** — 상세 비교는 [3.6절](#) 참고.

2. 읽기 경로 — 사용자가 메신저로 사진을 보낼 때

2.1 단계별 흐름

1. 사용자가 텔레그램 봇에 라벨 사진을 보낸다
2. Telegram이 POST /telegram-webhook 으로 Worker를 호출한다
Worker는 처리 작업을 waitUntil()에 맡기고 곧바로 200을 반환한다.
아래 세 가지는 그 뒤 백그라운드에서 진행된다.
 - 5MB 이하 중 가장 큰 해상도의 photo를 고른다
(전부 5MB를 넘으면 가장 작은 photo[0]으로 대체)
 - "라벨을 분석하고 있습니다... 📷" 안내 메시지를 보낸다
 - PHOTO_QUEUE.send({chatId, messageId, fileId})로 큐에 넣는다
3. Queue Consumer가 큐에서 1건씩 꺼낸다 (max_batch_size = 1)
 - getFile로 이미지를 내려받아 base64로 바꾼다
 - Gemini Vision으로 라벨 정보를 추출한다
 - 추출 결과를 검색 문자열로 조립한 뒤 임베딩한다
 - Supabase RPC search_products를 호출한다 (pgvector + 카테고리 필터)
 - 메타데이터로 재정렬하고 확신도를 판정한다 (필요하면 Gemini로 재검증)
4. sendMessage로 텔레그램에 HTML 포맷 결과를 회신한다

2단계에서 **Worker가 200을 먼저 반환한다는 점**이 중요합니다. 안내 메시지 발송과 큐 적재는 응답을 보낸 뒤에 일어나므로, 파이프라인이 아무리 길어져도 Telegram이 기다리는 시간에는 영향을 주지 않습니다(→ 2.4절).

2.2 데이터 변환 표

#	입력 형태	변환 주체	출력 형태
1	사진 (사용자 기기)	Telegram 서버	file_id 문자열 + 해상도별 photo[]
2	file_id	services/telegram.ts getFileUrl	다운로드 URL
3	URL	search.ts fetch + utils/encoding.ts	base64 문자열 (5MB 초과 시 거부)
4	base64 이미지	Gemini 2.5 Flash (Vision)	ExtractedLabelInfo JSON {productType, brand, brandKorean, brandEnglish, exporterEnglish, numbers, volume, region,
5	추출 JSON	buildSearchText()	단일 검색 문자열 사케: 브랜드 + 브랜드(KR) + 브랜드(EN) + 수출사 + rawText 와인: 와이너리 + 지역 + 품종 + 빈티지 + 브랜드 3종 + rawText (→ 3.6절)
6	검색 문자열	Gemini Embedding (gemini-embedding-001)	number[768]
7	벡터	Supabase RPC search_products	Product[] 최대 50건 (similarity > 0.5)
8	후보 배열	prioritizeByMetadata() (순수 TS, AI 아님)	재정렬된 후보 배열
9	상위 3건 + base64 이미지	Gemini Vision verifyMatch	{matchedIndex, confidence}
10	판정 결과	utils/formatter.ts	텔레그램 HTML 문자열

2.3 Triple Search — 3단 판정 로직

벡터 검색만으로는 오인식이 나기 때문에 세 겹의 필터를 겹칩니다 (services/search.ts).

1단 — 벡터 검색 + 카테고리 필터

Gemini가 판정한 productType 을 그대로 SQL 필터로 넘겨, 와인 사진이 사케 후보를 끌어오는 것을 막습니다. 다만 이 필터는 **주종마다 규칙이 다름**니다 (migration_wine_support.sql).

질의 카테고리	실제로 검색되는 범위
Wine	Wine + Etc-Wine — 좁게 한정
Spirits	Spirits + Etc-Spirits — 좁게 한정
Sake	Wine · Etc-Wine 을 뺀 전부 — 사케·소주·기타주류· Other 까지 포함

와인만 배타적으로 걸러내고, 사케 질의는 나머지를 폭넓게 훑는 **비대칭 구조**입니다. 일본 주류는 청주·소주·리큐르 경계가 싹고 단계에서 흐려지는 경우가 많아, 사케 쪽은 후보를 넓게 잡고 2·3단에서 걸러내는 편을 택한 것입니다.

2단 — 메타데이터 가산점 재정렬 (prioritizeByMetadata)

벡터 유사도만으로는 잡히지 않는 도메인 지식을 점수로 반영합니다.

주종	배점
와인	와이너리 완전일치 100 / 부분일치 80 / 제품명 포함 60 / 단어단위 20×n 지역 50, 품종 50, 빈티지 30
사케	수출사 완전일치 100 / 부분일치 80 / 제품명 포함 60 / 단어단위 20×n 숫자 매칭(예: "23", "50") 50

점수가 같으면 벡터 유사도 순으로 정렬합니다. 사케 쪽에는 안전장치가 하나 더 있어서, 모든 후보가 0점이면 재정렬을 포기하고 원래 벡터 순서를 그대로 씁니다. 근거 없는 점수로 순위를 흔드느니 벡터 판단을 믿는 편이 낫다는 판단입니다.

3단 — 이미지 재검증 또는 스킵 (High Confidence Skip)

아래 조건이면 Gemini 재호출 없이 즉시 확정합니다 — 응답 지연과 API 할당량을 아끼는 최적화입니다.

- 최상위 유사도 ≥ 0.82 , 또는
- 최상위 유사도 ≥ 0.75 이면서 1위-2위 격차 > 0.15

스킵 조건에 못 미치면 상위 3건과 원본 이미지를 다시 Gemini에 보내 육안 대조시킵니다.

Gemini는 **두 값을 함께** 돌려줍니다 — 몇 번 후보가 맞는지(`matchedIndex` , 없으면 0)와 얼마나 확신하는지(`confidence`)입니다. 응답은 **둘을 모두 보고** 갑니다.

<code>matchedIndex</code>	<code>confidence</code>	결과
<code>> 0</code>	≥ 70	확정 — "제품을 찾았습니다"
무관	50 ~ 69	보류 — "불확실합니다. 아래 제품이 맞는지 확인해주세요"
<code>0</code>	≥ 70	실패 — "수입 목록에서 찾을 수 없습니다" (후보만 제시)
무관	< 50	실패 — 위와 같음

`matchedIndex` 를 따로 보는 이유는, Gemini가 **"확실히 셋 다 아니다"** 라고 답할 수 있기 때문입니다. 이때 `matchedIndex: 0` , `confidence: 90` 같은 값이 나오는데, `confidence` 만 봤다면 90점짜리 확정으로 처리해 엉뚱한 제품을 답할 뻔합니다. 높은 확신도가 곧 일치를 뜻하지는 않습니다.

2.4 이 경로의 방어 장치

장치	값	이유
Queue <code>max_batch_size</code>	1	사진 여러 장 동시 전송 시 Gemini rate limit 회피
Queue <code>max_retries</code> / DLQ	3회 / <code>photo-search-dlq</code>	일시 장애 복구, 최종 실패 격리
Gemini 모델 폴백	2.5 Flash → 2.0 Flash	1차 모델 장애/한도 시 자동 전환
Vision 타임아웃	20초	Worker 실행시간 한도 초과 방지
Embedding 타임아웃	10초	위와 같음
벡터 검색 타임아웃	15초 (<code>Promise.race</code>)	DB 응답 지연 차단
라벨 추출 시도	총 2회 (<code>MAX_RETRIES = 2</code>)	첫 시도가 실패하면 한 번 더. 단, 추출 <code>confidence</code> ≥ 70 이면 두 번째 시도를 생략 (DB에 없는 제품으로 간주)
이미지 크기	5MB 상한	Worker 메모리 보호

웹훅이 200을 먼저 반환하는 이유: Telegram은 응답이 늦으면 같은 업데이트를 재전송합니다. 전체 파이프라인(Vision 20초 + Embedding 10초 + DB 15초 + 재검증)은 웹훅 응답 시간에 들어갈 수 없으므로, 웹훅은 접수만 하고 무거운 작업은 Queue Consumer로 넘깁니다.

3. 쓰기 경로 — 사이트에서 데이터를 넣을 때

3.1 단계별 흐름

1. 관리자가 Pages 사이트에서 비밀번호를 입력한다
- GET /admin/stats 가 200을 주면 통과로 본다
2. .xlsx 파일을 고른다
- SheetJS가 브라우저 안에서 파싱한다

('제품별_합산' 시트를 우선 찾고, 없으면 첫 시트를 쓴다)

• 'Product Name (KR)' 컬럼이 있는지 미리 확인한다
3. 업로드 모드를 고른다
- Smart (기본) — 기존 행은 UPDATE하고, 없는 행만 INSERT한다

• Reset — POST /admin/upload-init 로 테이블을 비운 뒤 전량 INSERT한다
4. 50행씩 묶어 최대 3개를 동시에 POST /admin/upload-chunk 로 보낸다
5. Worker(handlers/admin.ts)가 청크마다 다음을 수행한다
- 컬럼명의 공백을 정리하고, 필수 컬럼이 빈 행을 걸러낸다

• 표시명과 임베딩 텍스트를 만들고, 카테고리들을 분류한다

• 4필드 복합키로 기존 행인지 신규 행인지 판별한다

• 기존 행 → RPC bulk_update_sake_imports (임베딩을 만들지 않는다)

• 신규 행 → Gemini batchEmbedContents로 벡터를 만든 뒤 insert
6. 브라우저가 진행률을 갱신한다
- 실패한 청크는 백오프 재시도하고, 중단되면 그 지점부터 다시 이어간다

3.2 데이터 변환 표

#	입력 형태	변환 주체	출력 형태
1	.xlsx 바이너리	SheetJS (브라우저)	ExcelRow[] JSON — 서버에 파일 자체는 전송되지 않음
2	ExcelRow[]	admin/js/upload.js	50행 단위 JSON 청크, 3개 병렬 POST
3	청크 JSON	admin.ts 정규화	컬럼명 trim(), Product Name (KR) 빈 행 제거
4	행	이름 조립	displayName = "닷사이 23 (Dassai 23)" — KR + (EN) embeddingText = KR + EN + Exporter + Origin Country
5	Category + HS-CODE	getCategoryFromExcelData()	ProductCategory 7종 중 하나
6	displayName 목록	Supabase SELECT ... IN	기존 행의 4필드 조합 Map
7	행	복합키 대조	toUpdate[] / toInsert[] 분리
8	toUpdate[]	RPC bulk_update_sake_imports(JSONB)	갱신 행 수 (네트워크 왕복 1회)
9	toInsert[] 의 embeddingText[]	Gemini batchEmbedContents	number[768][]
10	행 + 벡터	supabase.from().insert()	sake_imports 신규 행

헛갈리기 쉬운 지점 — 임베딩은 브라우저가 아니라 Worker가 만듭니다.

브라우저가 하는 일은 xlsx를 JSON으로 바꿔 보내는 것까지입니다(1~2단계).

벡터 생성(9단계)은 Gemini API 키가 필요한데, 그 키는 Worker secret이라 브라우저에 내려주지 않습니다. 또한 벡터를 만드는 대상은 행 전체가 아니라 embeddingText 한 줄(제품명 KR + EN + 수출사 + 원산지)이며, 금액·물량 같은 숫자 컬럼은 벡터에 들어가지 않고 일반 컬럼으로만 저장됩니다.

검색 대상이 아니라 검색 결과로 보여줄 값이기 때문입니다.

3.3 카테고리 분류 규칙

주종을 잘못 넣으면 읽기 경로의 카테고리 필터가 통째로 어긋나므로, 한글 카테고리를 1순위로 쓰고 애매한 것만 HS-CODE로 보조 판정합니다.

Category = '과실주' → Wine

Category = '청주' → Sake

Category = '소주' | '일반증류주' → Spirits

Category = '기타주류' | '리큐르' → HS-CODE로 세분류

HS 2204xx → Etc-Wine

HS 2206xx → Etc-Sake

HS 2208xx → Etc-Spirits

그 외 → Other

Category = '탁주' | '소스' → Other

Category 미상 → HS-CODE 단독 판정 → 최종 폴백 Other

분류에 실패한 행도 Other 로 저장할 뿐 버리지 않습니다. 다만 **Other 가 검색에 걸리는 것은 사케 질의일 때뿐**입니다 — 와인 질의는 Wine / Etc-Wine 만 보기 때문에 Other 에 들어간 와인성 제품은 잡히지 않습니다(2.3절 1단).

Category 와 HS-CODE 를 정확히 채워야 하는 이유가 여기 있습니다.

3.4 중복 판정 — 4필드 복합키

key = 제품명 | Exporter | Origin Country | Raw Importer Name

제품명만으로 판정하면 같은 제품을 서로 다른 수입사가 들어온 실적이 하나로 뭉개집니다. 수입 이력 조화가 목적인 서비스에서 이는 데이터 손실이므로, 수출사·원산지·수입사까지 포함한 복합키로 구분합니다.

3.5 이 경로의 핵심 설계 결정

① 브라우저에서 파싱한다

엑셀 바이너리를 Worker로 보내지 않습니다. Worker의 요청 크기·CPU 시간 제약을 피하고, 파싱 비용을 클라이언트로 넘기는 구조입니다. 서버는 언제나 정제된 JSON만 받습니다.

② SQL 문자열을 만들지 않는다

브라우저도 Worker도 SQL을 조립하지 않습니다. Worker는 Supabase JS SDK를 쓰고, SDK는 이를 **PostgREST HTTP 호출**로 변환합니다. 실제 SQL은 database/*.sql 에 미리 정의해 둔 저장 함수뿐이고, Worker는 그것을 rpc() 로 호출만 합니다.

RPC 함수	호출 지점	역할
truncate_sake_imports()	/admin/upload-init	Reset 모드 전체 삭제 (DELETE보다 빠름)
bulk_update_sake_imports(JSONB)	/admin/upload-chunk	다건 UPDATE를 왕복 1회로
search_products(vector, count, threshold, category)	Queue Consumer	pgvector 유사도 검색
get_stats()	/admin/stats	총 건수·최종 갱신일·상위 수출사

bulk_update_sake_imports 가 이 설계의 이유를 잘 보여줍니다. 50행을 개별 UPDATE하면 왕복 50회지만, JSONB 배열 하나로 넘기면 1회입니다.

(저장 함수와 RPC가 무엇인지는 5.3절 참고)

③ 기존 행은 임베딩을 다시 만들지 않는다

임베딩 생성이 이 파이프라인에서 가장 비싼 단계입니다. 갱신되는 값은 금액·물량·단가뿐이고 제품명은 그대로이므로, toUpdate 경로는 Gemini를 아예 호출하지 않습니다. 재업로드 비용이 신규 행 수에만 비례합니다.

④ 실패해도 이어서 한다

재시도는 브라우저와 Worker 두 층에 각각 있고, 설정이 서로 다릅니다.

	브라우저 — 체크 전송 재시도 (admin/js/upload.js fetchWithRetry)	Worker — 임베딩 호출 재시도 (handlers/admin.ts retryWithBackoff)
최대 횟수	5회	3회
대기 간격	2s → 4s → 8s → 16s	2s → 4s (+ 0~500ms jitter)
재시도 안 하는 경우	4xx 응답 (그대로 반환)	unauthorized / invalid / forbidden 포함 오류

즉 일시 장애(5xx-네트워크)는 두 층에서 모두 버티고, 인증-검증 오류는 어느 층에서도 재시도하지 않습니다. 비밀번호가 틀렸는데 16초씩 기다리며 5번 두드리는 낭비를 막기 위해서입니다.

여기에 더해 브라우저가 lastProcessedIndex 를 들고 있어 중단 지점부터 재개할 수 있습니다. Gemini 일일 할당량에 걸려 업로드가 멈춰도 이미 처리한 분량은 날아가지 않습니다.

3.6 적재 텍스트 vs 조회 텍스트 — 의도된 비대칭

두 경로가 만나는 지점은 name_embedding 벡터 컬럼 하나입니다. 쓰기 경로가 텍스트를 768차원 벡터로 바꿔 넣고, 읽기 경로가 사진에서 뽑은 텍스트를 같은 공간으로 투영해 비교합니다. 그런데 양쪽이 조립하는 텍스트의 재료가 서로 다릅니다.

	적재 시 — embeddingText (handlers/admin.ts:145)	조회 시 — searchText (services/search.ts buildSearchText)
사케	제품명(KR) + 제품명(EN) + Exporter + Origin Country	브랜드 + 브랜드(KR) + 브랜드(EN) + Exporter + rawText
와인	(사케와 동일 — 주종 구분 없음)	Exporter(와이너리) 먼저 + region + grape + vintage + 브랜드 3종 + rawText
출처	엑셀 컬럼	Gemini Vision 추출 결과

차이가 세 가지입니다.

- 1. 적재 쪽은 주종을 구분하지 않습니다. 사케든 와인이든 같은 순서로 조립합니다.
조회 쪽만 와인일 때 와이너리를 맨 앞에 놓습니다.
- 2. rawText 는 조회 쪽에만 있습니다. 라벨에서 읽어낸 전체 텍스트라
도수·수상 문구 같은 노이즈가 함께 들어갑니다.
- 3. region / grapeVariety / vintage 는 DB 벡터에 없습니다.
엑셀에 해당 컬럼 자체가 없어서, 제품명 문자열에 우연히 포함된 경우에만 반영됩니다.

이 비대칭이 왜 문제가 되지 않는가.

벡터 공간이 일치해야 한다는 제약은 모델과 차원(gemini-embedding-001 , 768)에 걸리는 것이지, 입력 문장의 형식에 걸리는 것이 아닙니다. 그리고 이 시스템은 벡터 검색을 정답을 집어내는 도구가 아니라 후보 50건을 긁어오는 그물로만 씁니다 (similarity > 0.5 , LIMIT 50). 노이즈가 섞여 순위가 흔들려도 정답이 50위 안에만 들면 됩니다.

빠진 정보는 다음 단계에서 되찾습니다. 벡터에 담기지 않은 region-grape-vintage는

2.3절 2단 재정렬에서 제품명 문자열에 직접 대조해

가산점으로 반영합니다(지역 50점, 품종 50점, 빈티지 30점). AI가 아니라 순수 TypeScript 문자열 비교입니다.

```
Gemini 추출 {winery:"Chateau Margaux", region:"Bordeaux", vintage:"2018"}
|
├─ 1단 벡터 검색(그물)           → 후보 50건, 순위는 대략적
├─ 2단 메타데이터 문자열 가산점 → 순위 교정 (벡터가 놓친 정보 보강)
└─ 3단 유사도 ≥0.82 확정 / 아니면 Gemini에 사진 재대조
```

즉 벡터 검색의 정밀도 부족을 2·3단이 매우는 전제로 설계돼 있고, 그래서 양쪽 텍스트 레시피를 굳이 일치시키지 않았습니다.

주의: 임베딩 모델이나 차원을 바꾸면 DB의 기존 벡터를 전부 재생성해야 합니다. 텍스트 레시피는 달라도 되지만, 모델·차원은 반드시 같아야 하기 때문입니다. 실제로 이 프로젝트도 임베딩 모델을 한 번 교체했는데, 그때 3072차원을 768로 잘라 쓴 것이 바로 이 재생성을 피하기 위한 선택이었습니다 (경위는 5.6절 끝의 각주 참고).

4. 서비스 연결 방식

연결	방식	인증
Telegram → Worker	Webhook (setWebhook 으로 URL 등록, polling 아님)	없음 (아래 참고)
Worker → Telegram	Bot API HTTPS	TELEGRAM_BOT_TOKEN (Worker secret)
Pages → Worker	fetch + CORS (*.pages.dev , localhost 허용)	Authorization: Bearer <ADMIN_PASSWORD>
Worker → Supabase	@supabase/supabase-js → PostgREST HTTPS	SUPABASE_KEY (Worker secret)
Worker → Gemini	@google/generative-ai + REST	GEMINI_API_KEY (Worker secret)
Worker → Queue	런타임 바인딩 env.PHOTO_QUEUE	바인딩이 대신 처리 (아래 참고)

키가 노출되지 않는 이유: SUPABASE_KEY , GEMINI_API_KEY , TELEGRAM_BOT_TOKEN 은 전부 wrangler secret put 으로 Worker에만 주입됩니다. 브라우저가 아는 것은 Worker의 URL과 관리자 비밀번호뿐이며, Supabase에 직접 접근하지 않습니다.

바인딩이란: env.PHOTO_QUEUE 처럼 Cloudflare가 런타임에 꽂아 주는 객체입니다. Queues 자체는 분산 서비스라 내부적으로는 네트워크를 타지만, URL·API 키·인증 헤더를 코드에서 다룰 필요가 없습니다. 자격 증명이 코드나 secret에 등장하지 않는다는 점이 외부 API 호출과 다른 지점입니다.

알려진 취약점 — 웹훅에 검증이 없습니다. /telegram-webhook 은 요청이 정말 Telegram에서 왔는지 확인하지 않습니다. URL을 아는 사람은 누구나 가짜 update JSON을 POST해 봇이 임의의 chat_id 로 메시지를 보내게 하거나 Gemini 할당량을 소진시킬 수 있습니다. Telegram은 이를 막으라고 setWebhook 의 secret_token 옵션을 제공하며, 지정하면 매 요청에 X-Telegram-Bot-API-Secret-Token 헤더가 실려 옵니다. 현재 코드는 이 헤더를 검사하지 않습니다.

services/database.ts 는 Supabase 클라이언트를 WeakMap<Env, SupabaseClient> 로 캐싱해 같은 isolate 안에서 재사용합니다.

5. 데이터베이스 — 기초부터

이 절은 SQL을 잘 모르는 상태에서 읽을 수 있도록 개념부터 시작합니다. 이미 아는 내용이면 5.4 sake_imports 테이블로 건너뛰세요.

5.1 PostgreSQL이란

PostgreSQL(줄여서 "포스트그레스", Postgres)은 오픈소스 관계형 데이터베이스입니다. 데이터를 엑셀 시트처럼 행(row)과 열(column)로 이루어진 표(table) 로 저장하고, SQL이라는 질의 언어로 그 표를 다룹니다.

엑셀과 대응시키면 이렇습니다.

엑셀	PostgreSQL	이 프로젝트에서
통합 문서	데이터베이스	Supabase 프로젝트 1개
시트	테이블	sake_imports
열 제목	컬럼 (타입이 고정됨)	reported_product_name TEXT
한 줄	행	수입 실적 1건
필터/정렬	SELECT ... WHERE ... ORDER BY	검색
VLOOKUP	WHERE ... IN (...) + 인덱스	복합키 대조

결정적인 차이는 **타입이 강제된다**는 점입니다. value **NUMERIC** 이라고 선언한 컬럼에 문자열을 넣으면 DB가 거부합니다. 엑셀에서 숫자 칸에 "약 3천"이라고 적히는 사고가 구조적으로 막힙니다.

SQL 자체는 훨씬 방대하지만, **이 프로젝트가 실제로 쓰는 동작은 네 가지**뿐입니다.

```
SELECT    ...  -- 읽기      (조회)
INSERT    ...  -- 넣기      (신규 제품 추가)
UPDATE    ...  -- 고치기    (기존 제품의 금액/수량 갱신)
TRUNCATE  ...  -- 비우기    (Reset 모드에서 테이블 전체 삭제)
```

앞의 셋은 데이터를 다루는 기본 동작(여기에 **DELETE** 를 더한 넷을 흔히 **CRUD**라 부릅니다)이고, **TRUNCATE** 는 조건 없이 테이블을 통째로 비우는 별도 명령입니다. **DELETE** 처럼 행을 하나씩 지우지 않아 훨씬 빠른 대신 되돌릴 수 없습니다. 테이블-인덱스를 만드는 **CREATE** , 함수를 정의하는 **CREATE FUNCTION** 등은 `database/*.sql` 에서 최초 설정 때 한 번씩만 쓰입니다.

Supabase는 PostgreSQL을 직접 설치·운영하지 않아도 되게 감싼 관리형 서비스입니다. DB 서버 자체는 순수한 PostgreSQL이고, Supabase가 그 위에 **PostgREST**라는 계층을 얹어 줍니다. PostgREST는 테이블을 자동으로 HTTP API로 노출하는 도구로, 덕분에 Worker에서 SQL 문자열을 조립하지 않고도 DB를 다룰 수 있습니다.

```
// Worker가 쓰는 코드
await supabase.from('sake_imports').select('reported_product_name').in('reported_product_name', names)

// PostgREST가 내부에서 실행하는 SQL
// SELECT reported_product_name FROM sake_imports WHERE reported_product_name IN (...)
```

즉 **SQL이 사라진 게 아니라, SDK가 대신 만들어 줍니다**. 이 구조의 이득은 SQL 문자열을 손으로 잊지 않으니 SQL 인젝션 위험이 없고, TypeScript 타입이 그대로 붙는다는 것입니다.

SDK와 PostgREST — 헛갈리는 두 이름

SDK(Software Development Kit)는 남이 만든 서비스를 내 언어에서 편하게 쓰라고 제공하는 코드 묶음입니다. 이 프로젝트가 `npm install` 로 받은 `@supabase/supabase-js` , `@google/generative-ai` 가 그것입니다. SDK가 없으면 HTTP를 직접 조립해야 합니다.

```
// SDK 없이 — URL 규칙과 헤더를 직접
await fetch('https://xxx.supabase.co/rest/v1/sake_imports?select=*&category=eq.Wine', {
  headers: { apikey: KEY, Authorization: `Bearer ${KEY}` }
})

// SDK로
await supabase.from('sake_imports').select('*').eq('category', 'Wine')
```

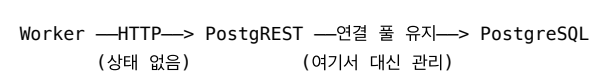
SDK는 편의 계층일 뿐이고, 내부에서 하는 일은 결국 `fetch` 입니다.

PostgREST는 그와 별개로, **PostgreSQL**을 자동으로 **REST API**로 바꿔주는 독립 프로그램입니다. Supabase가 만든 것이 아니라 오픈소스이고, Supabase가 이를 가져다 각 프로젝트 앞단에 세워 둡니다. 핵심은 *자동*이라는 점 — 테이블이나 함수를 만들면 API 코드를 한 줄도 쓰지 않아도 엔드포인트가 생깁니다.

DB에 만든 것	자동으로 생기는 엔드포인트
sake_imports 테이블	GET/POST /rest/v1/sake_imports
search_products() 함수	POST /rest/v1/rpc/search_products
get_stats() 함수	POST /rest/v1/rpc/get_stats

이 구조가 이 프로젝트에 잘 맞는 이유가 있습니다. 일반적인 DB 연결은 TCP 소켓을 열어 두고 재사용하는 방식이라, 접속 비용을 여러 쿼리에 나눠 갖는 **연결 풀**을 전제로 합니다. 그런데 Cloudflare Workers는 요청마다 뒀다 사라지고 전 세계 수많은 지점에서 동시에 실행되므로, 풀을 유지할 주체가 없고 오히려 접속 수만 폭증하기 쉽습니다.

HTTP는 요청 하나가 그 자체로 완결되어 이 문제를 겪지 않습니다. PostgREST가 DB를 HTTP로 감싸 준 덕분에 Worker는 평범한 `fetch` 한 번으로 DB를 다룰 수 있고, 연결을 관리할 일이 없습니다.



Workers에서 PostgreSQL에 TCP로 직접 붙는 것이 불가능하지는 않습니다. `cloudflare:sockets` 의 `connect()` 나 Hyperdrive(연결 풀링·캐싱을 대신해 주는 Cloudflare 서비스를 쓰면 됩니다. 이 프로젝트가 PostgREST 경로를 택한 것은 Supabase가 기본으로 제공하는 방식이고 추가 설정이 필요 없기 때문입니다.

5.2 pgvector — "비슷함"을 SQL로 물어보기

PostgreSQL은 **확장(extension)** 으로 기능을 덧붙일 수 있습니다. `CREATE EXTENSION vector;` 한 줄이면 **pgvector**가 설치되고, 그때부터 `vector` 라는 새로운 컬럼 타입과 유사도 연산자를 쓸 수 있게 됩니다.

왜 필요한가. 일반 SQL 검색은 글자가 맞아야 찾습니다.

```
SELECT * FROM sake_imports WHERE reported_product_name LIKE '%닷사이%';
```

라벨 사진에서 "DASSAI"를 읽어냈는데 DB에 "닷사이"로 저장돼 있으면 이 쿼리는 아무것도 못 찾습니다. "쿠보타"와 "久保田", "Kubota"도 마찬가지입니다. 글자가 아니라 **의미가 비슷한 것**을 찾아야 하는데, 그게 `LIKE` 로는 불가능합니다.

임베딩(embedding)이 그 해법입니다. AI 모델에 텍스트를 넣으면 숫자 배열이 나옵니다. 이 프로젝트에서는 768개짜리 실수 배열입니다.

```
"닷사이 23 (Dassai 23) Asahi Shuzo Japan"
  ↓ Gemini Embedding
[0.021, -0.184, 0.077, ... ] ← 768개
```

이 숫자 배열을 **768차원 공간의 한 점**이라고 보면, 핵심 성질은 이것입니다.

의미가 비슷한 텍스트는 서로 가까운 점이 된다.

"닷사이"와 "Dassai"는 겹치는 글자가 하나도 없어 `LIKE` 로는 서로를 절대 찾지 못하지만, 임베딩하면 두 점이 거의 같은 자리에 놓입니다. 표기 체계가 달라도 모델이 같은 대상을 가리킨다고 학습했기 때문입니다. 그래서 **글자 비교를 거리 계산으로 바꾸면** 언어와 표기가 달라도 찾을 수 있습니다.

이 "점"을 저장할 컬럼 타입과 거리 계산을 함께 제공하는 것이 pgvector입니다.

name_embedding halfvec(768) -- 768개 숫자를 담는 컬럼

거리를 재는 연산자도 pgvector가 함께 추가합니다. 세 종류가 있고, 이 프로젝트는 코사인 거리 <=> 를 씁니다.

연산자	의미
<=>	코사인 거리 — 이 프로젝트가 쓰는 것
<->	유클리드(직선) 거리
<#>	내적에 음수를 붙인 값 (작을수록 유사)

코사인 거리는 두 점이 원점에서 뻗어나가는 방향의 차이만 봅니다. 크기(문장 길이, 단어 수)는 무시하고 의미의 방향만 비교하므로 텍스트 검색에 적합합니다.

- 거리 0 = 방향이 완전히 같음 = 의미가 같음
- 거리 1 = 방향이 90도 = 무관
- 거리 2 = 정반대

사람이 읽기 편하도록 이 프로젝트는 유사도로 뒤집어서 씁니다.

```
1 - (name_embedding <=> query_embedding) AS similarity
-- 유사도 1.0 = 동일, 0.5 = 어중간, 0.0 = 무관
```

search_products 함수의 뼈대가 이런 모양입니다. 읽기 쉽도록 컬럼 나열과 카테고리 조건을 줄였으니, 실제 본문은 database/migration_wine_support.sql 을 보시기 바랍니다.

```
SELECT
    si.*, -- (실제로는 컬럼을 하나씩 나열)
    1 - (si.name_embedding <=> query_embedding) AS similarity -- ① 유사도 계산
FROM sake_imports si
WHERE 1 - (si.name_embedding <=> query_embedding) > similarity_threshold -- ② 0.5 미만 버림
    AND (category_filter IS NULL OR si.category = category_filter) -- ③ 주종 필터
ORDER BY si.name_embedding <=> query_embedding -- ④ 가까운 순
LIMIT match_count; -- ⑤ 최대 50건
```

실제 함수가 이보다 더 하는 일은 두 가지입니다. 하나는 맨 앞에서 SET LOCAL hnsw.ef_search = 100 으로 인덱스 탐색 깊이를 지정하는 것이고 (→ 5.5절), 다른 하나는 ③의 주종 필터가 위치럼 단순 일치가 아니라 Wine · Sake · Spirits 별로 갈라지는 OR 체인이라는 점입니다(→ 2.3절 1단).

앞에서 본 임계값들(0.5, 0.82, 0.75)이 전부 이 similarity 값입니다. ORDER BY 에 유사도가 아니라 거리 <=> 를 그대로 쓴 것은 인덱스를 타기 위해서입니다(→ 5.5절).

5.3 저장 프로시저와 RPC

RPC(Remote Procedure Call, 원격 프로시저 호출)는 다른 컴퓨터에 있는 함수를 내 컴퓨터의 함수처럼 호출하는 방식을 가리킵니다. 일반 REST가 "자원"을 다루는 데 비해(테이블에서 행을 읽고 쓰기), RPC는 "동작"을 부릅니다.

```
// 자원 지향 - "이 테이블에서 조건에 맞는 행 줘"
supabase.from('sake_imports').select('*').eq('category', 'Wine')

// RPC - "이 함수를 실행해줘"
supabase.rpc('search_products', { query_embedding, match_count: 50 })
```

그리고 그 "원격에 있는 함수"의 정체가 **저장 프로시저(stored function)** 입니다.

DB 안에 미리 저장해 둔 함수로, CREATE FUNCTION 으로 정의하면 이름으로 호출할 수 있습니다.

```
CREATE OR REPLACE FUNCTION truncate_sake_imports()
RETURNS void AS $$
BEGIN
    TRUNCATE TABLE sake_imports;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

await supabase.rpc('truncate_sake_imports'); // Worker에서 이렇게 호출
```

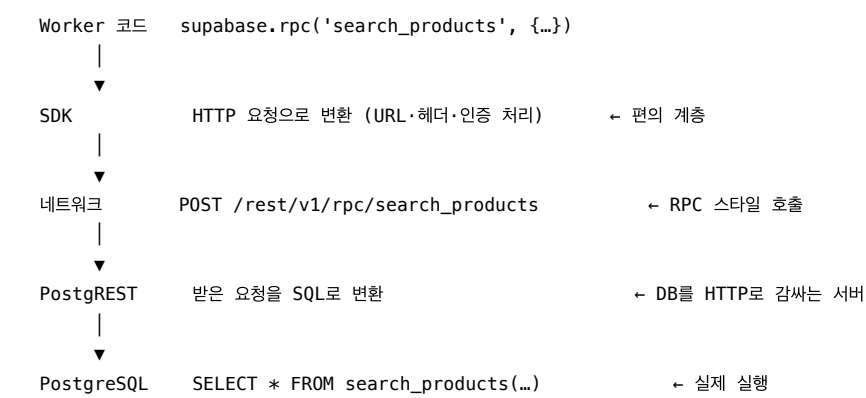
왜 이렇게 하는가. 이유가 세 가지입니다.

- 1. **PostgREST로 표현할 수 없는 연산** — <=> 같은 벡터 연산자는 REST 문법으로 표현할 방법이 없습니다. search_products 가 함수여야만 하는 이유입니다.
- 2. **네트워크 왕복 절감** — bulk_update_sake_imports 는 50건의 UPDATE를 JSONB 배열 하나로 받아 DB 안에서 반복합니다. 왕복 50회가 1회가 됩니다.
- 3. **권한 제어** — SECURITY DEFINER 는 "호출자 권한이 아니라 함수 소유자 권한으로 실행하라"는 뜻입니다. 익명 키로도 TRUNCATE가 가능해지는 대신, 그 권한이 **이 함수 안에서만** 유효합니다.

\$\$... \$\$ 는 함수 본문을 감싸는 인용 부호이고, plpgsql 은 PostgreSQL의 절차형 언어(변수·반복·조건문이 되는 SQL 확장)입니다.

한 줄이 지나가는 네 개의 층

지금까지 나온 SDK · RPC · PostgREST · PostgreSQL이 어떻게 맞물리는지는, Worker 코드 한 줄을 따라가 보면 한눈에 정리됩니다.



세 용어를 한 줄로 구분하면 이렇습니다.

용어	정체	어디에 있나
SDK	내 코드가 쓰는 라이브러리	Worker 코드에 함께 번들되어 배포됨
RPC	함수를 부르는 호출 방식	네트워크를 오가는 규약
PostgREST	DB 앞에 선 번역 서버	Supabase 인프라

5.4 sake_imports 테이블

컬럼	타입	비고
id	BIGSERIAL	PK

컬럼	타입	비고
reported_product_name	TEXT	"한글명 (English Name)" 형식
category	TEXT	ProductCategory 7종
exporter	TEXT	사케는 양조장, 와인은 와이너리
origin_country , raw_importer_name	TEXT	복합키 구성 요소
value , volume , unit_price	NUMERIC	재업로드 시 갱신되는 값
name_embedding	halfvec(768)	검색 키
created_at , updated_at	TIMESTAMP TZ	

부속 테이블: upload_progress (업로드 세션 상태), search_logs (검색 이력).

5.5 벡터 인덱스 — 왜 필요하고, 왜 HNSW인가

인덱스가 없으면, 사진 한 장을 검색할 때마다 DB가 **전체 행과 하나씩 거리를 계산**해야 합니다. 2,500건이면 768차원 거리 계산을 2,500번 수행하는 셈입니다. 데이터가 10만 건으로 늘면 10만 번이 되고, 그쯤이면 사용자는 붓이 멈춘 줄 압니다.

벡터 인덱스는 이 문제를 "정확도를 조금 포기하고 속도를 크게 얻는" 방식으로 푼니다. 이를 **ANN**(Approximate Nearest Neighbor, 근사 최근접 이웃) 검색이라 부릅니다. 전수 비교가 아니므로 **이론상 진짜 1등을 놓칠 수 있지만**, 실제로는 거의 항상 맞고 수십~수백 배 빠릅니다.

이 프로젝트는 IVFFlat에서 시작해 HNSW로 옮겼습니다 (database/migration_hnsw.sql).

	IVFFlat	HNSW (현재)
원리	벡터를 100개 클러스터로 나눠, 질의와 가까운 클러스터만 탐색	벡터를 다층 그래프로 잇고, 이웃을 따라 목표 쪽으로 이동
약점	클러스터 경계에 걸친 항목을 통째로 놓침	메모리를 더 쓰고 인덱스 구축이 느림
소량 데이터	클러스터당 표본이 적어 불안정	안정적

전환 이유는 속도가 아니라 **재현율**이었습니다. IVFFlat은 정답이 탐색하지 않은 클러스터에 들어 있으면 아예 후보에 오르지 못하는데, 수천 건 규모에서는 이 사고가 드물지 않습니다. 검색 실패가 곧 서비스 실패인 구조라 안정성을 택했습니다.

```
CREATE INDEX idx_sake_imports_embedding
ON sake_imports USING hnsw (name_embedding vector_cosine_ops)
WITH (m = 24, ef_construction = 128);
-- 검색 시: SET LOCAL hnsw.ef_search = 100;
```

여기 등장하는 파라미터 세 개는 각각 이런 뜻입니다.

파라미터	값	의미	올리면
m	24	그래프에서 노드 하나가 갖는 연결 수	정확 ↑, 메모리 ↑
ef_construction	128	인덱스를 만들 때 탐색 깊이	정확 ↑, 구축 시간 ↑
ef_search	100	인덱스로 검색할 때 탐색 깊이	정확 ↑, 검색 시간 ↑

앞의 둘은 인덱스에 굳어지고, ef_search 만 쿼리마다 바꿀 수 있습니다. 그래서 search_products 함수가 실행될 때 SET LOCAL 로 지정합니다 (LOCAL = 이 트랜잭션에서만 적용).

vector_cosine_ops 는 "이 인덱스는 코사인 거리(<=>) 기준으로 만든다"는 선언입니다. 인덱스의 기준 연산자와 쿼리의 연산자가 다르면 인덱스를 타지 못하고

전수 검색으로 되돌아갑니다. ORDER BY 에 유사도(1 - ...)가 아니라 거리 <=> 를 그대로 쓴 이유가 이것입니다 — 계산식으로 감싸면 인덱스가 무시됩니다.

halfvec — 정밀도를 절반으로 줄인 벡터 타입.
벡터 하나가 768개의 숫자이므로 저장 비용이 만만치 않습니다. 기본 vector 타입은 숫자 하나를 32비트(4바이트)로 담아 행당 약 3KB를 쓰는 반면, halfvec 은 16비트 (2바이트)로 담아 행당 약 1.5KB, **정확히 절반**을 씁니다. Supabase 무료 티어 용량 안에 들어가기 위한 선택입니다.

정밀도는 유효숫자 기준으로 약 7자리에서 3자리 남짓으로 떨어집니다. 다만 이 시스템이 벡터에서 얻으려는 것은 정확한 거리값이 아니라 **후보의 순서**이고, 그 뒤로도 메타데이터 재정렬과 이미지 재검증이 남아 있어 이 정도 손실은 결과를 바꾸지 않는다고 보고 택했습니다. (pgvector 0.7.0 이상 필요)

5.6 SQL 적용 순서

schema.sql 은 초기 버전이라 IVFFlat 인덱스와 category_filter 없는 search_products 가 들어 있습니다. 현재 구조를 재현하려면 순서대로 적용해야 합니다.

- 1. schema.sql 기본 테이블 + 함수
- 2. migration_hnsw.sql IVFFlat → HNSW, search_products 재정의
- 3. migration_wine_support.sql category_filter 파라미터 추가 (와인 지원)
- 4. migration_truncate.sql truncate_sake_imports()
- 5. bulk_update_function.sql bulk_update_sake_imports()
- 6. add_composite_index.sql 복합 인덱스
- 7. fix_unique_constraint.sql / fix_warnings.sql

임베딩 모델은 gemini-embedding-001 이며, 원래 3072차원 출력을 outputDimensionality: 768 로 축소해 사용합니다 (services/gemini.ts:7). 초기에 쓰던 text-embedding-004 가 API 키에서 지원되지 않아 교체한 것으로 (자세한 경위는 CHANGELOG.md 2026-01-29 항목), 차원을 768로 맞춰 스키마 변경 없이 전환했습니다.

6. 주변 장치

Cron Keepalive — wrangler.toml 의 crons = ["0 */6 * * *"] 로 6시간마다 cron/keepalive.ts 가 sake_imports count 쿼리를 1회 실행합니다. Supabase 무료 티어의 비활성 프로젝트 일시정지를 막기 위한 것으로, 첫 사용자 요청이 콜드 DB를 깨우느라 지연되는 상황을 방지합니다.

Smart Placement — [placement] mode = "smart" . Cloudflare가 Worker 실행 위치를 자동 배치합니다. 이 워크로드는 요청 1건당 Gemini와 Supabase를 여러 번 왕복하므로, 사용자 근처보다 외부 API 근처에서 실행되는 편이 총 지연이 짧습니다.

에러 알림 — utils/logger.ts 의 logErrorAndNotify 가 ADMIN_CHAT_ID 가 설정된 경우 관리자 텔레그램으로 오류를 전달합니다.

7. 요약 — 두 흐름 한눈에 비교

	읽기 경로 (조회)	쓰기 경로 (적재)
트리거	사용자의 사진 전송	관리자의 엑셀 업로드
진입점	POST /telegram-webhook	POST /admin/upload-chunk
인증	없음 — 웹훅 검증 미구현 (4절)	Bearer 비밀번호

	읽기 경로 (조회)	쓰기 경로 (적재)
비동기	Cloudflare Queues (1건씩)	브라우저 측 3병렬 청크
AI 사용	Vision 추출 + 임베딩 + Vision 재검증	임베딩만 (신규 행 한정)
DB 연산	RPC search_products (읽기)	RPC bulk_update_sake_imports + insert (쓰기)
변환 축	이미지 → 텍스트 → 벡터 → 행 → 메시지	엑셀 → JSON → 텍스트 → 벡터 → 행
지연 목표	수 초 (사용자 대기)	수 분 (배치, 재개 가능)
실패 처리	재시도 3회 → DLQ → 사용자 안내	백오프 재시도 → 중단 지점 재개